

COURSE WORKFLOW GUIDE

Unity + GitHub Desktop

Organise, version, protect, and move your course work

For beginners working independently across one or more computers.

This guide uses one course repository containing design notes, a proposal, three Unity prototypes, and three evaluations.

Semester 2, 2026

Guide version 1.0

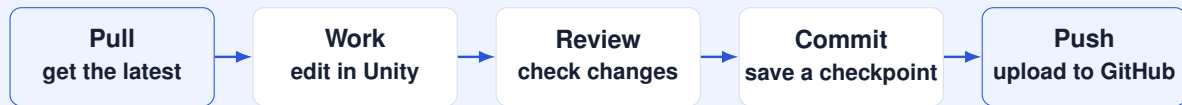
Contents

1 Quick reference	2
2 What GitHub is doing	3
3 Before you start	3
3.1 Create a GitHub account	3
3.2 Install GitHub Desktop and sign in	3
3.3 Install Unity Hub and the Editor	3
4 Create the course repository	4
5 Install the supplied .gitignore	4
6 Create your first Unity project	5
7 Your first commit	5
7.1 Review what Git can see	5
7.2 Commit and publish	5
8 Organising the course repository	6
8.1 What belongs in each folder	6
8.2 Add them and commit	6
9 What to keep out of Git	7
10 The daily workflow	7
10.1 Start of a session	7
10.2 During a session	8
10.3 End of a session	8
11 Moving between computers	8
11.1 Setting up a computer that has never had the repository	8
12 Common problems	9
Appendix: authoritative references	9

1 Quick reference

This page is a summary. If you are setting up for the first time, start at Section 3 and come back here once your repository is working.

The daily workflow



Three essential rules

1. Pull before beginning work.
2. Commit meaningful checkpoints.
3. Push before you leave or switch computers.

Large-file rule

Normal GitHub repositories block individual files larger than 100 MiB. Keep builds, long recordings, archives, caches, and other large generated files outside the repository unless your course explicitly uses Git LFS.

Before beginning

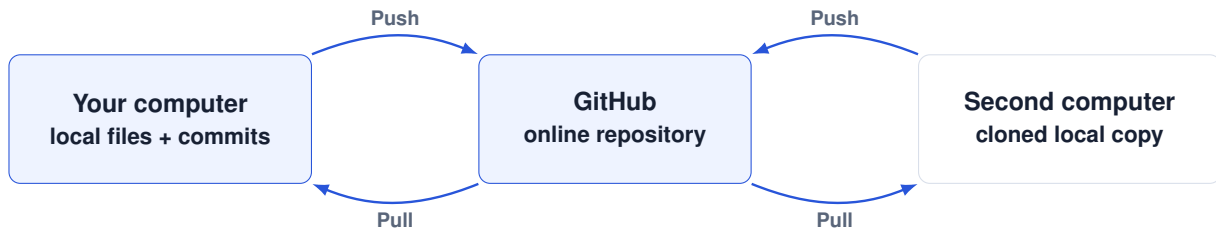
- ☐ Open GitHub Desktop first
- ☐ Fetch origin
- ☐ Pull any available changes
- ☐ Open the correct prototype
- ☐ Confirm no unresolved conflicts

Before finishing

- ☐ Save and close Unity
- ☐ Review changed files
- ☐ Exclude large/generated files
- ☐ Commit with a useful summary
- ☐ Push origin
- ☐ Confirm repository is up to date

2 What GitHub is doing

GitHub gives your work a history. A **commit** saves a named checkpoint on your computer. A **push** sends those commits to the online repository. A **pull** retrieves newer commits from GitHub.



Term	Meaning
Repository	A project folder plus its Git version history.
Commit	A named checkpoint saved in the local repository.
Push	Upload local commits to GitHub.
Fetch	Check GitHub for newer commits without changing local files.
Pull	Download and apply newer commits from GitHub.
Clone	Download a repository to a computer for the first time.

Sections 3 to 7 are done once

Sections 3–7 set everything up and finish with your first commit. After that you repeat the short loop in Section 10 for the rest of the course.

3 Before you start

Install these three things and sign in before creating anything. Publishing a repository later will not work until you are signed into GitHub Desktop.

3.1 Create a GitHub account

1. Go to <https://github.com> and create a free account if you do not already have one.
2. Use an email address you will keep for the whole course.
3. Verify that email address. GitHub will not let you publish work until you do.

3.2 Install GitHub Desktop and sign in

1. Download GitHub Desktop from <https://desktop.github.com> and install it.
2. Open it and sign in with the GitHub account you just created.
3. Check that your name and email are filled in, under **File** > **Options** > **Git** on Windows, or **GitHub Desktop** > **Settings** > **Git** on macOS. These appear on every commit you make.

3.3 Install Unity Hub and the Editor

1. Install Unity Hub and sign in with a Unity account.
2. Install the Unity Editor version specified for the course, and use that version for every prototype.

Use the version the course specifies

Opening a project in a newer Editor upgrades it, and the upgrade cannot be undone by pulling. If you and a lab computer use different versions, the project can break when work moves between them.

4 Create the course repository

You will create the repository first, as an empty folder, and then build everything else inside it.

1. Open GitHub Desktop.
2. Select **File** > **New repository**.
3. For **Name**, enter XR-YourName. This name becomes the folder name.
4. For **Local path**, choose where the folder should live, such as your Documents folder. GitHub Desktop creates the repository folder inside that location.
5. Leave **Git ignore** set to None. You will install the supplied course file in Section 5.
6. Select **Create repository**.

You now have one folder that is a Git repository. Everything for the course goes inside it.

One repository, at the top

Create this repository once. Do not create a second repository inside Prototype1, inside Assets, or for each prototype, unless your teaching team explicitly instructs you to.

5 Install the supplied .gitignore

Download the supplied .gitignore and place it directly inside the repository folder you just created, before you create any Unity project.

A .gitignore tells Git which files to leave alone. Unity generates a very large amount of temporary data that it can rebuild at any time, and none of it belongs on GitHub.

What this file prevents

Without it, opening a Unity project inside the repository would leave GitHub Desktop showing something like:

```
Library/Bee/artifacts/...      ~ several thousand changed files
Library/PackageCache/...
Library/ShaderCache/...
Temp/...
Logs/...
```

None of that is your work. With the file installed, you will see three folders instead.

Check the filename

✓ **Correct:** .gitignore ✗ **Incorrect:** .gitignore.txt

On Windows, enable file-name extensions in File Explorer before checking. The leading dot is part of the filename.

The supplied version is designed for a repository containing several nested Unity projects. It recursively ignores generated Unity folders, IDE metadata, builds, logs, test output, archives, and common operating-system files.

6 Create your first Unity project

The first prototype is created **inside** the repository folder, in a subfolder named Prototype1. The full course layout comes later, in Section 8.

1. Open Unity Hub and select **New project**.
2. Choose the Editor version and template required by the course.
3. Set **Project name** to Prototype1.
4. Set **Location** to your repository folder, so the project is created at XR-YourName/Prototype1.
5. Create the project and wait for Unity to finish its first import.
6. Confirm that Prototype1 now contains Assets, Packages, and ProjectSettings.

Inside, not beside

If Unity Hub creates the project next to the repository folder rather than inside it, Git cannot see the project at all. Check the path before selecting **Create project**.

7 Your first commit

This section is the loop you will repeat for the rest of the course. Do it once now, with a small amount of work, so the steps are familiar.

7.1 Review what Git can see

Switch back to GitHub Desktop. It should list a short set of changes: the .gitignore and the three Unity folders that matter.

Expected changes

.gitignore Prototype1/Assets/ Prototype1/Packages/ Prototype1/ProjectSettings/

This short list is the .gitignore doing its job. Without it, the same screen would be listing several thousand generated files.

Stop if generated files appear

If GitHub Desktop shows thousands of files inside Library, do not commit. The .gitignore is missing, incorrectly named, or in the wrong folder. Fix it before going any further — removing these files after they are committed is much harder.

7.2 Commit and publish

1. Enter a summary describing what you did, such as Set up course repository and Prototype 1.
2. Select **Commit to main**.
3. Select **Publish repository**.
4. Keep the repository private unless the course explicitly requires a public one.
5. Open the repository on <https://github.com> and confirm your files are there.

You have now done the whole loop

Review the changes, commit them with a useful summary, push them to GitHub. That is the loop, and it does not get more complicated than this. Section 10 covers when to run it.

8 Organising the course repository

Your repository currently holds one prototype. Add the remaining folders now so there is an obvious place for every piece of work.

```

XR-YourName/
|-- README.md
|-- .gitignore
|-- DesignNotes/
|   |-- Notes/
|   |-- Sketches/
|   `-- Images/
|-- Proposal/
|-- Prototype1/
|   |-- Assets/
|   |-- Packages/
|   `-- ProjectSettings/
|-- Evaluation1/
|-- Prototype2/      (same three folders)
|-- Evaluation2/
|-- Prototype3/      (same three folders)
`-- Evaluation3/
  
```

8.1 What belongs in each folder

DesignNotes	Raw notes, ideas, sketches, screenshots, observations, useful links, and working thoughts from across the course. This is flexible working material, not a separate journal.
Proposal	The initial design proposal and any supporting material required for it.
Prototype1–3	Each folder is a complete, self-contained Unity project. Its root contains Assets, Packages, and ProjectSettings.
Evaluation1–3	The testing plan, results, and supporting evidence for the prototype immediately before it. Keep this explanation brief and follow the assessment instructions for the required contents.
README.md	A short identification page containing the course, student name, and a summary of the repository.
.gitignore	Rules telling Git which generated, temporary, build, and local files must not be tracked.

8.2 Add them and commit

1. Create DesignNotes, Proposal, and the numbered evaluation folders inside the repository.
2. Add a short README.md naming the course and yourself.
3. Create Prototype2 and Prototype3 in Unity Hub when the course reaches them, using the same method as Section 6.
4. Run the loop again: review, commit with a summary such as Add course folder structure, and push.

Git does not track empty folders

A folder with nothing in it will not appear in GitHub Desktop. This is normal. It will show up as soon as you put a file in it.

9 What to keep out of Git

Large-file mistakes are among the easiest ways to make a beginner repository difficult or impossible to push. Check file size **before** committing large media or exported applications.

GitHub's normal file limit

GitHub warns when a normal Git repository contains a file larger than 50 MiB and blocks files larger than 100 MiB. Git LFS can store files beyond the normal limit, but it introduces quotas and should only be used when the course requires it.

Item	Track?	Reason
Assets	Yes	Scripts, scenes, prefabs, materials, models, and required project content.
Packages	Yes	Records the packages required by the project.
ProjectSettings	Yes	Records Unity project configuration.
Design notes and proposal	Yes	Part of the course work and its history.
Testing plans and results	Yes	Part of each evaluation record.
Small evidence images	Yes	Genuinely part of the project record.
Library	No	Large generated cache that Unity recreates.
Temp, Logs	No	Temporary and diagnostic files.
Builds and exported apps	No	Large generated outputs that can be rebuilt.
ZIP archives and backups	No	Whole-project copies do not belong in a repository that already keeps history.
Duplicated or unused media	No	High-resolution source files nothing references.
Package installers	No	Platform and build artefacts.
Long videos and raw footage	Usually no	Store in the location specified in your assessment task sheet.

10 The daily workflow

10.1 Start of a session

1. Open GitHub Desktop before Unity.
2. Select the course repository.
3. Select **Fetch origin**.
4. If GitHub Desktop reports remote changes, select **Pull origin**.
5. Only then open the required prototype in Unity Hub.

Why fetch comes first

Fetch checks whether GitHub contains newer commits. **Pull** downloads and applies them. Starting with this check prevents you from editing an older copy of the project.

10.2 During a session

Commit at meaningful checkpoints. A commit should describe a change you may later want to identify or restore.

Commit summaries**✓ Useful**

Add teleport interaction
Build first menu layout
Fix grabbing on small objects
Prepare Prototype 2 for testing

✗ Unhelpful

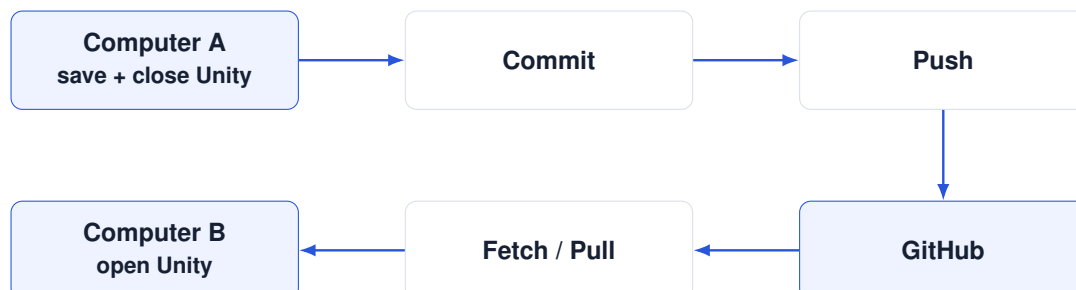
stuff
changes
final final
asdf

10.3 End of a session

1. Save your Unity scenes and project.
2. Close Unity so it finishes writing project files.
3. Review the changed files in GitHub Desktop.
4. Confirm that no large, generated, or unrelated files are included.
5. Write a meaningful commit summary.
6. Select **Commit to main**.
7. Select **Push origin**.
8. Confirm GitHub Desktop reports that the branch is up to date.

11 Moving between computers

Finish and push on one computer, then pull on the other. The steps are the ordinary start and end of a session from Section 10.

**11.1 Setting up a computer that has never had the repository**

1. Install GitHub Desktop, Unity Hub, and the required Unity Editor version, as in Section 3.
2. Sign into GitHub Desktop.
3. Select **Clone a repository** and choose the course repository.

4. Choose a suitable local location.
5. In Unity Hub, add the required Prototype1, Prototype2, or Prototype3 folder as a project.
6. Allow Unity to recreate its generated Library files. The first opening may take time.

Do not work on both copies at once

Do not keep the same project open and make independent edits on two computers. Finish, commit, and push on one computer before pulling and continuing on the other.

12 Common problems

Publish repository does not work

Check that you are signed into GitHub Desktop and that you have verified your GitHub email address. Both are covered in Section 3.

My latest work is missing

Check whether the previous computer committed **and pushed** the work. Then fetch and pull on the current computer. A local commit that was never pushed cannot appear on another device.

Unity is importing everything

This is expected after cloning or opening the project on a new computer. Unity is recreating the ignored Library folder.

GitHub Desktop shows thousands of generated files

Stop before committing. Check that the file is named exactly `.gitignore` and sits at the top of the repository, beside Prototype1 rather than inside it.

A file is too large to push

Do not repeatedly retry. Remove the file from the commit and move it to the location specified in your assessment task sheet. Adding a filename to `.gitignore` after it has already been committed does not automatically remove it from Git history; ask the teaching team for help.

GitHub reports a merge conflict

Stop editing and do not choose conflict-resolution options at random. Preserve a copy of your current work and ask the teaching team for help.

Appendix: authoritative references

- GitHub's maintained Unity ignore template:
<https://github.com/github/gitignore/blob/main/Unity.gitignore>
- GitHub documentation for large files:
<https://docs.github.com/en/repositories/working-with-files/managing-large-files>
- Git LFS and GitHub Desktop:
<https://docs.github.com/en/desktop/configuring-and-customizing-github-desktop/about-git-large-file-storage-and-github-desktop>